State-of-the-Art Analyse und Vorhersage von SPL-Token: Die Yellowstone gRPC Geyser und ShredStream Architektur

1. Einleitung und Paradigmenwechsel in der Solana-Datenarchitektur

Im Jahr 2026 hat sich die Solana-Blockchain mit einem verarbeiteten dezentralen Börsenvolumen (DEX) von über 1,95 Billionen US-Dollar und mehr als 33 Milliarden Transaktionen jährlich zur dominanten Infrastruktur für hochfrequenten On-Chain-Handel (High-Frequency Trading, HFT) und algorithmische Finanzapplikationen entwickelt.[1] Die extrem kurzen Blockzeiten von durchschnittlich 400 Millisekunden sowie der massive Transaktionsdurchsatz erfordern von Marktteilnehmern, Arbitrage-Systemen und Machine-Learning-Pipelines eine technologische Anpassung, die traditionelle Blockchain-Architekturen fundamental infrage stellt.[3] In dieser Umgebung ist die Latenz nicht länger ein reines Optimierungsziel, sondern die primäre Determinante für die Profitabilität algorithmischer Strategien und die Vorhersagegenauigkeit von Preismodellen für SPL-Token (Solana Program Library).[2]

Historisch verließen sich Entwickler und Analysten auf Standard-JSON-RPC-Polling-Methoden (wie getAccountInfo oder getProgramAccounts) oder native WebSocket-Abonnements (accountSubscribe, logsSubscribe), um den Zustand der Blockchain abzufragen.[3] Diese Architektur stößt in modernen Hochlastszenarien jedoch an eine physische und konzeptionelle Grenze, die in der Industrie als "RPC Wall" bezeichnet wird.[3] Beim klassischen Polling vergehen oft mehrere Hundert Millisekunden, bis ein HTTP-Request vom Node verarbeitet und beantwortet wird.[3] Auf einem Netzwerk, in dem der Slot-Leader alle vier Slots (ca. 1,6 Sekunden) wechselt, bedeutet eine Latenz von 200 Millisekunden, dass die abgerufenen Daten bereits beim Eintreffen in der Applikationslogik veraltet sind.[3] Arbitrage-Fenster sind geschlossen, Liquidierungen wurden bereits von kompetitiven Akteuren durchgeführt und der Markt hat sich weiterbewegt.[7]

Auch das offizielle PubSub-System über WebSockets bietet in der Praxis nur eine Illusion von Echtzeit.[7] Private Nutzerberichte und offizielle Benchmarks belegen, dass WebSockets bei Lastspitzen – etwa während massiver Liquidationskaskaden oder bei hochvolumigen Memecoin-Launches auf Plattformen wie Pump.fun – rapiden Degradierungen unterliegen.[7] Der Overhead der JSON-Serialisierung auf Node-Seite führt zu einem rechenintensiven

Subscription-Fan-Out.[7] Da WebSockets keine integrierte Backpressure-Kontrolle besitzen, resultiert dies in verworfenen Logs, Slot-Race-Conditions durch außer der Reihe eintreffende Events und schwer wiegenden Reconnect-Schleifen.[7] Ein Trading-Bot oder ein prädiktives Modell, das während der volatilsten und profitabelsten Marktphasen blind wird, verliert seinen gesamten statistischen Vorteil.[4]

Die absolute State-of-the-Art (SOTA) Lösung für die Analyse und Vorhersage von SPL-Token-Bewegungen verabschiedet sich daher vollständig vom JSON-RPC- und WebSocket-Standard.[7] Sie nutzt stattdessen eine tief in die Validator-Architektur integrierte Push-Streaming-Methode: das **Yellowstone gRPC Geyser Plugin**, zwingend kombiniert mit **Jito ShredStream** auf dedizierten Bare-Metal-Servern, und verarbeitet diese Datenströme durch Allokations-freie Rust-Pipelines sowie fortschrittliche Deep-Learning-Modelle (insbesondere Gated Recurrent Units).[7] Der vorliegende Bericht analysiert diese SOTA-Architektur, untersucht offizielle Dokumentationen von Infrastruktur-Providern wie Chainstack sowie detaillierte Feldberichte und Code-Repositories aus der Open-Source-Community, um zu demonstrieren, wie das volle Potenzial dieser Technologien zur SPL-Coin-Analyse ausgeschöpft wird.

2. Die technologische Basis: Solana Geyser und Yellowstone gRPC

Um den Engpass der herkömmlichen RPC-Schnittstellen zu umgehen, implementierten die Solana Labs das Geyser-Plugin-System.[3] Dieses System erlaubt es Validatoren, den massiven RPC-Load auszulagern, indem Echtzeitinformationen über Accounts, Transaktionen, Slots und Blöcke direkt in externe Datenspeicher oder Streaming-Plattformen gepusht werden.[12] Geyser operiert auf der niedrigsten Ebene des Validator-Speichers und extrahiert Datenstrukturen, noch bevor diese den Konsensprozess vollständig durchlaufen haben oder auf Festplatten persistiert wurden.[11]

2.1 Dragon's Mouth: Die Yellowstone gRPC Implementierung

Die am weitesten verbreitete und leistungsfähigste Implementierung dieses Geyser-Interfaces ist "Dragon's Mouth", allgemein bekannt als Yellowstone gRPC, entwickelt und gewartet von Triton One.[15] Es handelt sich hierbei um eine in Rust geschriebene dynamische Shared Library (.so-Datei), die beim Start in den Speicherbereich des Solana-Validators geladen wird.[3] Das Plugin registriert Callbacks (Hooks) direkt an der AccountsDb und der Bank des Validators.[3] Sobald der Validator eine Transaktion verarbeitet, triggert der Callback, und Dragon's Mouth greift die Rohdaten ab.[3]

Der entscheidende Vorteil von Yellowstone gRPC liegt in der Datenübertragung.[11] Anstatt Daten als rechenintensive Base64-encodierte JSON-Strings zu versenden, nutzt Yellowstone das gRPC-Framework (Google Remote Procedure Call) über HTTP/2.[6] Die Daten werden

deterministisch in kompakte, stark typisierte Protocol Buffers (Protobufs) serialisiert.[6] Dies minimiert die Payload-Größe massiv und reduziert den Bandbreitenbedarf.[18] HTTP/2 bringt zudem natives Multiplexing und Flow-Control (Backpressure-Management) mit sich, was bedeutet, dass der Stream auch bei extremem Durchsatz stabil bleibt und der Server die Übertragungsrate an die Verarbeitungskapazität des Clients anpassen kann, ohne Pakete blindlings zu verwerfen.[3]

2.2 Leistungsvergleich der Datenprotokolle

Die Überlegenheit von Yellowstone gRPC lässt sich anhand der Latenzmetriken und architektonischen Eigenschaften klar quantifizieren.

Metrik	**Standard RPC (Polling)**	**Native RPC WebSockets**	**Yellowstone gRPC (Dragon's Mouth)**
Latenz (Slots)	p90 ~150 ms	p90 ~10 ms	p90 ~5 ms
Latenz (Accounts)	Abhängig von Polling-Rate	~374 ms	~215 ms
Payload-Format	Hoch (JSON + Base64)	Mittel (JSON + Base64)	Minimal (Protobuf Binärformat)
Zuverlässigkeit	Mittel (Rate Limits, Retries)	Niedrig-Mittel (fragile Verbindungen)	Hoch (Native HTTP/2 Flow Control)
Datenstruktur	Manuelles Parsing notwendig	Manuelles Parsing notwendig	Typisiert, strukturierte Felder
Filter-Logik	Client-seitig, hoher Overhead	Grobgranular, unflexibel	Server-seitig, feingranular

Tabelle 1: Gegenüberstellung der Performance-Metriken und Protokolleigenschaften bei der Solana-Datenextraktion.[3]

Ein Bericht der Entwickler-Community verdeutlicht den Unterschied: Während WebSockets oftmals keine Historie bieten und Subscription-Limits (meist 100-200 Subscriptions pro Verbindung) den Aufbau von massiven Überwachungsnetzwerken für tausende SPL-Token verhindern, skaliert Yellowstone horizontal.[6] Nutzer können exakte Filter definieren – etwa nur Transaktionen zu erhalten, die nicht fehlgeschlagen sind (failed = False) und die mit dem Pump.fun-Programm (6EF8rrecthR5Dkzon8Nwu78hRvfCKubJ14M5uBEwF6P) interagieren.[15] Da das Filtern serverseitig (innerhalb des Geyser-Plugins) abläuft, erreicht die Applikationslogik nur der Datenstrom, der tatsächlich analytischen Wert besitzt.[21]

2.3 Datenströme und Commitment-Level

Yellowstone gRPC bietet verschiedene Stream-Typen, die je nach Anwendungsfall der SPL-Token-Analyse kombiniert werden können [21]:

- **Accounts**: Überträgt Echtzeit-Updates von Token-Balances und Programm-Statusänderungen. Optimal für das Tracking von Wallet-Bewegungen (Whale-Watching).[21]
- **Transactions**: Liefert vollständige Transaktionsdetails. Essenziell für Swap-Tracker, Arbitrage-Bots und das Erfassen von Programm-Aufrufen.[21]
- **Transaction_Status**: Eine leichtgewichtige Variante, die lediglich über Erfolg oder Fehlschlag (confirmed/failed) informiert.[21]
- **Slots & Blocks**: Nützlich für zeitbasierte Logik, Taktung (Pacing) und die Sortierung von Events in der Datenbank.[21]

Ein weiterer kritischer Parameter ist das Commitment-Level. Für absolut zeitkritische HFT- und Machine-Learning-Features wird das Level PROCESSED gewählt, um Updates sofort nach der lokalen Verarbeitung durch den Leader zu erhalten, auch wenn der Konsens noch nicht netzwerkweit finalisiert ist.[15]

3. Infrastrukturelle Exzellenz: Jito ShredStream und Hardware-Kolokation

Die bloße Verwendung von Yellowstone gRPC löst das Serialisierungsproblem, eliminiert jedoch noch nicht alle Latenzen auf der physikalischen Netzwerkschicht. Der "RPC Wall" wird durch zwei zusätzliche infrastrukturelle Maßnahmen durchbrochen: Jito ShredStream und kompromissloses Hardware-Tuning.[8]

3.1 Die Rolle von Jito ShredStream

Auf der Solana-Blockchain generiert der Validator (Slot-Leader) kontinuierlich Blöcke. Diese Blöcke werden in atomare, verifizierbare Pakete fester Größe unterteilt, die sogenannten "Shreds" (Fragments of a block).[3] Diese Shreds werden normalerweise über das "Turbine"-Protokoll (ein mehrschichtiges Gossip-Netzwerk) an die restlichen Validatoren im Netzwerk propagiert.[3] Ein Standard-RPC-Node empfängt diese Shreds erst, nachdem sie durch mehrere Hops im Turbine-Tree gewandert sind, was die Latenz um 15 bis 80 Millisekunden erhöhen kann.[3]

Die SOTA-Lösung umgeht das Turbine-Protokoll vollständig durch den Einsatz von **Jito ShredStream**.[3] ShredStream nutzt das Jito Block Engine Netzwerk, um Shreds direkt vom Slot-Leader an dedizierte Empfänger (wie den eigenen Trader Node) auszuliefern, sobald diese generiert werden.[3] Dies eliminiert die physikalischen Netzwerk-Hops.[3]

Ermittlerberichte und Benchmarks aus der Entwickler-Community (z. B. von Dysnix) belegen die massive Überlegenheit dieser Architektur. In einem Test über 2.078.707 abgeglichene Transaktionen lieferte ShredStream die Daten in 64,5 % der Fälle schneller als Yellowstone gRPC ohne ShredStream.[4] Der durchschnittliche Zeitvorteil betrug 32,8 Millisekunden, wobei in Spitzenzeiten (maximale Latenz-Abweichung) ein Vorsprung von bis zu 1.323 Millisekunden gemessen wurde.[4] Ein Vorteil von 32 Millisekunden mag gering klingen, entscheidet in der Solana-Arbitrage jedoch darüber, ob ein Copy-Trade im exakt selben Slot landet oder einen Slot zu spät eintrifft, wodurch die Arbitrage-Gelegenheit wertlos wird.[4] Eine korrekte Architektur leitet daher den ShredStream als primäre Signalquelle direkt in das Geyser-Plugin ein, welches die rohen Shreds verarbeitet und als sauberen gRPC-Stream ausgibt.[24]

3.2 Hardware-Spezifikationen und Kernel-Bypass (DPDK)

Um die hereinkommenden Datenmengen (oft zehntausende Updates pro Sekunde) verarbeiten zu können, scheitern virtuelle Maschinen in geteilten Cloud-Umgebungen an CPU-Stealing und Memory-Bandbreiten-Konflikten.[5] Die absolut fortschrittlichste Lösung basiert auf dedizierten Bare-Metal-Servern, die sich im selben Rechenzentrum (Kolokation) wie die größten Solana-Validator-Cluster befinden (typischerweise in Frankfurt, New York, Amsterdam oder Tokio).[8]

Aus Hardware-Perspektive definieren sich SOTA Trader-Nodes im Jahr 2026 durch folgende Parameter [5]:

Komponente	Minimum-Spezifikation (Produktion)	Empfohlene SOTA-Spezifikation	Begründung in der Solana-Architektur

Prozessor (CPU)	16+ physische Kerne (x86_64)	24–32 Kerne, AMD EPYC (z. B. 9355/7443P)	Hohe Taktrate und massiver L3-Cache sind kritisch für Single-Thread-Performance.[5]
Arbeitsspeicher	256 GB DDR4/5 ECC	512 GB – 1,5 TB DDR5 ECC	Accounts müssen vollständig im RAM gecacht werden, um langsames Disk-Thrashing zu verhindern.[8]
Massenspeicher	2 TB NVMe SSD (Gen4)	4 TB+ NVMe SSDs in RAID-0/1 Konfiguration	Bewältigung des extremen I/O-Aufkommens der Ledger-Speicherung.[30]
Netzwerkanbindung	1 Gbps dediziert	10 Gbps redundante Uplinks mit DPDK	Vermeidung von Bottlenecks während Congestion-Phasen.[30]

Tabelle 2: Hardwarespezifikationen für hochleistungsfähige Solana gRPC Nodes.[5]

Auf der Ebene des Betriebssystems implementieren elitäre Trading-Desks Kernel-Bypass-Technologien wie das **Data Plane Development Kit (DPDK)** oder tiefgreifende eBPF-Filter.[4] Der Standard-Linux-Netzwerkstack ist für Universalzwecke konzipiert und kostet bei jedem Paket wertvolle Mikrosekunden durch Kontextwechsel (Context Switches) zwischen Kernel- und Userspace.[33] DPDK umgeht den Linux-Kernel, weist die Netzwerkkarte (NIC) direkt der Applikation zu und verarbeitet Pakete durch aggressives Busy-Polling der Empfangs-Warteschlangen (RX Queues) im Userspace.[32] Dies senkt den Netzwerk-Jitter signifikant und hält die p99-Latenz selbst während massiven Spam-Attacken auf das Netzwerk flach.[4]

4. Plattform-Integration: Das Ökosystem von Chainstack

Der Aufbau und die Wartung eines eigenen Bare-Metal-Validators mit DPDK und

Geyser-Plugins ist extrem ressourcenintensiv. Das Entpacken von Snapshots, die ständige Synchronisierung und die operative Fehlerbehebung lenken von der eigentlichen quantitativen Modellierung ab.[11] Provider wie **Chainstack** haben sich 2026 als SOTA-Infrastrukturpartner etabliert, indem sie die Komplexität abstrahieren und Enterprise-Grade-Zugang zu Yellowstone gRPC und ShredStream in einem Managed-Service-Modell anbieten.[19]

4.1 Spezifikationen und Preismodelle

Offiziellen Chainstack-Quellen zufolge ist das Yellowstone gRPC Geyser Plugin als Add-on für Global Nodes und Dedicated Nodes im Solana-Mainnet verfügbar.[19] Ein essenzielles Differenzierungsmerkmal der Chainstack-Infrastruktur ist, dass Jito ShredStream standardmäßig und ohne zusätzliche Konfiguration auf den Knoten aktiviert ist, wodurch die oben genannten Latenzvorteile sofort greifen.[19]

Das Zugangsmodell von Chainstack wurde 2026 dergestalt optimiert, dass es nicht nur für Institutionen, sondern auch für ambitionierte private Nutzer finanzierbar ist.[21] Die Abonnementstruktur für das gRPC-Plugin gliedert sich in folgende Tiers [19]:

- **$49/Monat:** Unbegrenzte Stream-Events (keine Volumengrenze), 1 paralleler Stream pro Account, Tracking von bis zu 50 Solana-Accounts pro Stream.[19]
- **$149/Monat:** Unbegrenzte Stream-Events, 5 parallele Streams, Tracking von bis zu 50 Solana-Accounts pro Stream.[19]
- **$449/Monat:** Unbegrenzte Stream-Events, 25 parallele Streams, Tracking von bis zu 50 Solana-Accounts pro Stream.[19]

Unabhängig vom gewählten Plan erlaubt jede Verbindung bis zu 5 gleichzeitige Filter desselben Typs.[19] Dies steht im starken Kontrast zu Providern wie Helius, die zwar mit "LaserStream" (ihrer eigenen gRPC-Lösung) exzellente Latenzen und historische Replay-Funktionen über 6000 Slots anbieten, jedoch ein Credit-basiertes Volumenmodell verfolgen.[1] Ein volumetrisches Modell kann bei Machine-Learning-Agenten, die unvorhersehbare und burst-artige Datenmengen generieren, zu massiven Kostenschwankungen führen.[35] Chainstacks Flat-Fee-Ansatz für unlimitierte Events (kombinierbar mit dem "Unlimited Node" Add-on) schützt die Ökonomie der Analyse-Pipeline vor exponentiellen Kostensteigerungen.[34]

4.2 Best Practices für die gRPC-Verbindung auf Chainstack

Die Integration des Chainstack Yellowstone gRPC Streams in Analyseumgebungen folgt strengen Best Practices, um Stabilität und Latenz zu optimieren.

1. **Verbindungsstabilität (Keep-Alive):** Load-Balancer trennen HTTP/2-Streams oftmals bei vermeintlicher Inaktivität.[3] Die offizielle Implementierung sieht vor, dass der Server alle 15 Sekunden ein asynchrones Ping-Signal sendet.[3] Der Client muss darauf zwingend mit einem Parameter (z. B. ping: true in der SubscribeRequest) und einem entsprechenden Pong antworten, um den Stream am Leben zu erhalten.[3]

2. **Bündelung (Connection Efficiency):** Anstatt hunderte Verbindungen für einzelne Token aufzubauen, müssen Analystemodelle multiple Adressen (bis zu 50 bei Chainstack) in einem einzigen Subscribe-Request ("Clubbing") aggregieren, um Thread-Erschöpfung auf Clientseite und Verbindungsabbrüche beim Provider zu vermeiden.[19]
3. **Fehlertoleranz durch from_slot:** Verbindungsabbrüche sind auf verteilten Systemen unvermeidlich.[3] Anstatt die Fehleranfälligkeit durch komplexe Redundanz- und Backfill-Logiken zu kompensieren, nutzen Entwickler den Parameter from_slot.[11] Bricht die Verbindung ab, übermittelt der Client beim Reconnect die letzte bekannte Slot-Nummer. Der Server spult daraufhin die verpassten Events aus dem Cache nach, was eine lückenlose Konsistenz für die Datenbank des Machine-Learning-Modells gewährleistet ("State-Consistency").[11]

5. Client-Architektur: Deserialisierung und Lock-Free Concurrency in Rust

Der am häufigsten dokumentierte Fehler von Entwicklern beim Aufbau von Geyser-Pipelines – intensiv diskutiert in Plattformen wie Reddit und StackExchange – ist die Wahl der falschen Programmiersprache für den gRPC-Client.[39] Nutzerberichte zeigen auf, dass Clients, die in Node.js (TypeScript/JavaScript) geschrieben sind, bei hohem Netzwerkdurchsatz nicht in der Lage sind, die Daten in Echtzeit zu konsumieren.[39] Node.js ist Single-Threaded; die asynchrone Ereignisschleife (Event Loop) und die ständigen Unterbrechungen durch den Garbage-Collector führen dazu, dass der Client die Nachrichten minimal langsamer verarbeitet (z. B. nur 99,9 % Geschwindigkeit), als der Server sie produziert.[39] Diese Diskrepanz summiert sich auf, erzeugt Speicherdruck und zwingt den Server letztendlich, die Verbindung wegen "Staleness" zu kappen.[39]

Die absolute SOTA-Lösung basiert vollständig auf **Rust** oder ersatzweise Go.[39] Rust bietet Kontrolle auf Systemebene ohne Garbage Collector und ist prädestiniert für High-Frequency-Trading.[39]

5.1 Multi-Threading und Lock-Free Queues

Ein performanter Rust-Consumer für Yellowstone gRPC vermeidet das Blockieren des Ingest-Threads. Daten, die aus dem Socket gelesen werden, dürfen nicht im selben Thread dekodiert werden. SOTA-Systeme implementieren Lock-Free Datenstrukturen (z. B. ArrayQueue Ringpuffer über Crates wie crossbeam oder Frameworks wie sof), um die gRPC-Events allokationsfrei (zero-allocation) in einen Consumer-Thread zu übergeben.[43] Diese Multi-Producer-Multi-Consumer (MPMC) Architektur trennt Netzwerk-I/O von der Business-Logik und verhindert Backpressure in Richtung des Solana-Nodes.[44]

5.2 Yellowstone Vixen: Die Revolution der Payload-Dekodierung

Die über gRPC empfangenen Transaktionen enthalten kompakte, binär kodierte Anweisungen (Instruction Data), die lediglich als Base58-Strings vorliegen.[45] Traditionell war das manuelle Parsen und Deserialisieren dieser Arrays fehleranfällig und aufwendig.[45]

Im Jahr 2026 hat sich **Yellowstone Vixen** als Standard-Framework zur Lösung dieses Problems etabliert.[46] Vixen ist ein Open-Source Rust-Framework für den Aufbau programm-bewusster Solana-Pipelines.[45] Es implementiert eine Parser-Handler-Architektur.[46] Das Herzstück von Vixen ist das prozedurale Makro include_vixen_parser!. Dieses Tool generiert zur Kompilierzeit automatisch typsichere Parser direkt aus den Codama JSON IDL-Dateien (Interface Definition Language) der jeweiligen Solana Smart Contracts.[46]

Das System besitzt vordefinierte Parser für das klassische SPL Token Program (TokenkegQfeZyi...) sowie für die moderneren Token-2022 Extensions (TokenzQdBNbLq...), die neue Metadaten-Pointer direkt im Mint-Account verankern, anstatt sie auf externe Programme auszulagern.[46] Vixen handhabt zudem "Discriminator Collisions" – Fälle, in denen verschiedene Instruktions-Varianten die gleichen 8-Byte Identifikatoren aufweisen – durch anpassbare InstructionResolver, die Varianten anhand der Anzahl der referenzierten Accounts sicher unterscheiden.[46] Durch Vixen können komplexe DeFi-Transaktionen (z.B. Raydium- oder Orca-Swaps inklusive Cross-Program-Invocations) latenzfrei und typsicher verarbeitet werden.[40]

6. Analyse, Extraktion und Feature Engineering für SPL-Token

Die Grundlage für prädiktive Modelle liegt im exakten Feature Engineering. Krypto-Märkte, insbesondere das Ökosystem der SPL-Memecoins, verzeihen keine fehlerhafte Datenberechnung.

6.1 Präzise Liquiditäts- und Preisberechnung

Ein weit verbreiteter Fehler unter Entwicklern ist der Versuch, den Preis eines SPL-Tokens durch den Vergleich von Pre- und Post-Token-Balances einer Transaktion zu ermitteln.[48] Dieses Vorgehen führt zu signifikanten Diskrepanzen ("Dusting"), da Arbitrage-Trades, komplexe Multi-Hop-Routen und Gebührenabzüge die Bilanzen verzerren.[48]

Die methodisch korrekte, absolute SOTA-Praxis (unter Verwendung von Tools wie Vixen oder Carbon) iteriert durch die geparste Transaktion und identifiziert die isolierten, statusverändernden Transferinstruktionen *innerhalb* der Liquiditätspools.[48] Das Skript extrahiert gezielt die Deltas zwischen dem Base-User-Vault und dem Quote-Vault (z.B. WSOL) des AMMs (Automated Market Maker).[48] Der exakte Preis eines Trades ergibt sich zwingend aus dem Quotienten des WSOL-Deltas und des Token-Deltas.[48] Diese saubere Datenextraktion ist zwingend erforderlich, um Features für Machine-Learning-Modelle wie exakte

Orderbuch-Tiefe und Mikrostruktur-Slippage ohne Rauschen ("Dust") bereitzustellen.[48]

6.2 Whale Tracking: Signale durch Netzwerktopologie

Ermittlungsberichte von Blockchain-Forensikern und Quant-Analysten zeigen, dass pure Price-Action-Daten unzureichend sind.[49] Ein drastischer Anstieg der Käufer-Wallet-Zahlen auf Dashboards wird oft durch "Fake Holder Inflation" manipuliert.[50] Die wahre Alpha liegt in der Identifizierung von Positionierungsverschiebungen (Whale Positioning), die der Preisbewegung stets *vorausgehen*.[49] "Die Divergenz zwischen dem Retail-Angst-Index und der Positionierung der Wale ist real... Die Positionierungsverschiebung kommt vor dem Preis", analysiert ein Nutzer anhand von aggregierten Daten.[49]

Um diese Verschiebungen systematisch aufzuspüren, nutzen SOTA-Ermittler den gRPC-Stream, um sogenannte **"Whale Cluster"** in Echtzeit zu tracken.[50] Die Strategie konzentriert sich auf die "1-Hop-Verteilungs-Heuristik".[50] Großinvestoren oder Projekt-Entwickler, die Positionen verschleiern wollen, transferieren große Mengen von SPL-Token von einer zentralen Deployer-Wallet an dutzende, scheinbar unverbundene Sekundär-Wallets (1-Hop), um diese Token von dort aus gestaffelt in Liquiditätspools zu verkaufen ("Stealth Selling").[50] Mehrere Hops (2-3 Jumps) werden selten genutzt, um Rechenaufwand und Gebühren zu minimieren.[50]

Mittels Yellowstone gRPC abonnieren Analysten gezielt Updates auf das SPL-Token Programm (programSubscribe).[51] Werden große, koordinierte Transfers ohne Gegenwert (0 SOL) an unberührte Wallets identifiziert, generiert das System einen "Net-Flow Imbalance"-Indikator.[49] Dieser Indikator signalisiert dem ML-Modell ein hohes Abverkaufsrisiko, noch bevor das Volumen auf den DEX-Plattformen sichtbar wird.[50] Eine solche Graph-Analyse ist nur durch die enorme Bandbreite und die fehlenden Limitierungen (unlimited events) von gRPC realisierbar.[19]

7. Prädiktive Vorhersage durch Advanced Machine Learning

Die extrahierten, bereinigten Datenströme (Preise, Volumen, Net-Flow Imbalance, Whale-Akkumulation) müssen in Echtzeit in Vorhersagemodelle eingespeist werden, um Marktbewegungen von SPL-Token zu prognostizieren.[52]

7.1 Schwächen von klassischen Modellen

Wissenschaftliche Analysen zur Preisvorhersage auf Kryptomärkten belegen konsistent, dass lineare Modelle wie ARIMA (Auto-Regressive Integrated Moving Average) oder klassische Support Vector Regression (SVR) aufgrund der extremen Non-Stationarität, Volatilität und der komplexen Marktdynamik von Kryptowährungen scheitern.[52] Auch standardmäßige Random Forests stoßen bei der temporalen Abhängigkeitsmodellierung im Hochfrequenzbereich an ihre

Grenzen, wenngleich sie für Anomalie-Erkennungen nützlich bleiben.[53]

7.2 Gated Recurrent Units (GRU) als SOTA-Standard

Die unangefochtene Benchmark für das Krypto-Zeitreihen-Forecasting auf Minute-zu-Minute-Basis und darunter bilden spezifische Architekturen rekurrenter neuronaler Netze (RNNs).[9] Studien evaluieren wiederholt Long Short-Term Memory (LSTM) Netze gegen **Gated Recurrent Units (GRU)**.[9] Die empirische Evidenz beweist, dass GRU-Modelle aufgrund ihres effizienteren internen Gating-Mechanismus (fehlender separater Zellzustand) nicht nur rechnerisch schneller inferenzieren, sondern LSTMs auch in der Vorhersagepräzision signifikant übertreffen.[9]

Ein optimiertes, zweischichtiges Deep-Learning-Netzwerk basierend auf GRU, trainiert mit einem Adam-Optimizer und einer Dropout-Rate von 0,2 zur Vermeidung von Overfitting (basierend auf einem 80:20 Train-Test-Split historischer Blockchain-Daten), erreichte in experimentellen Benchmarks herausragende Werte.[9] Die Modelle verzeichneten einen Mean Absolute Percentage Error (MAPE) von lediglich **0,09 %** (und in Krypto-spezifischen Analysen zwischen 0,035 und 0,087), was einer fast perfekten Prognosegenauigkeit des Preistrends für den nächsten Zeitraum entspricht.[9]

7.3 Integration von Makro-Faktoren via DBN

Da SPL-Memecoins stark vom Krypto-Makroumfeld abhängen, wird das GRU-Modell häufig durch einen multivariaten Ansatz wie das **Dynamic Bayesian Network (DBN)** flankiert.[52] Studien zeigen, dass neben technischen On-Chain-Indikatoren (die wichtigsten Prädiktoren) auch die Preisbewegung von Bitcoin, Social-Media-Sentiment und Nachrichten einen kausalen Einfluss auf Altcoin-Preisbewegungen haben.[52] Das DBN modelliert diese komplexen, bedingten Wahrscheinlichkeiten und passt das Basis-Risiko des Modells an das globale Sentiment an, bevor das GRU-Netzwerk den spezifischen SPL-Token evaluiert.[52]

7.4 Daten-Infrastruktur für Modelltraining

Machine-Learning-Agenten benötigen Zugang zur tiefen Historie und zum Echtzeit-Stream simultan. Ein Antipattern in diesem Bereich ist es, ShredStream oder Geyser als direkt abfragbare Datenbank zu missbrauchen.[10] Dies führt zu schmerzhaften Backfills und Ausfällen während Marktturbulenzen.[10]

Die Referenzarchitektur implementiert eine Event-Driven Data Pipeline:

1. **Message Queue:** Die durch Yellowstone gRPC/Vixen geparsten Events werden als Append-Only-Log an performante Message Broker wie Kafka oder Redpanda übergeben.[10]
2. **Analytische Datenbank (ClickHouse):** ClickHouse fungiert als Storage-Schicht für schnelle zeitbasierte Aggregationen.[56] Das Datenmodell umfasst eine Raw-Events-Tabelle

(ENGINE = MergeTree()), partitioniert nach Monat (PARTITION BY toYYYYMM()) und geordnet nach Programm-Typ und Timestamp, sowie Materialized Views (ENGINE = SummingMergeTree()), die im Hintergrund automatisch stündliche OHLC-Kerzen und Metriken für das ML-Training berechnen.[57]

3. **Historical Replay:** Für das Bootstrapping neuer Modelle wird "Old Faithful" (eine historische Komponente des Yellowstone-Stacks) verwendet.[3] Es ermöglicht das Replay des gesamten Ledger-Archivs als gRPC-Stream, sodass das Modell auf exakt derselben Schnittstelle trainiert wird, die es auch in der Live-Produktion nutzt.[3]

8. HFT-Bots und Ausführungsschicht (Sniper Bots)

Das beste prädiktive Signal ist nutzlos, wenn die Transaktionsobjektion nicht ausgeführt wird. Entwickler-Repositories von Trading- und Sniper-Bots (wie solana-arbitrage-bot oder Solana-Pumpfun-Sniper-Bot) offenbaren die architektonischen Muster, die die Top 10% der profitablen Trader vom Rest trennen.[58]

Ein SOTA Solana Trading-Bot arbeitet nach dem Schema: **Discovery → Quote → Fee Check → Execution**.[58] Die *Discovery* erfolgt primär über den gRPC Stream (das oben diskutierte Signal).[58] Ein Trigger, etwa die Erstellung eines neuen SPL-Tokens auf Pump.fun oder ein lukratives Liquiditätsdreieck, löst den Algorithmus aus.[59]

8.1 Transaktionsausführung und MEV-Schutz

Für die *Execution* verzichten professionelle Bots auf Standard-RPC-sendTransaction-Aufrufe. Transaktionen, die blind in den öffentlichen Mempool gesendet werden, fallen unweigerlich **Sandwich-Attacken** und Front-Running durch MEV-Bots (Maximal Extractable Value) zum Opfer.[61]

Stattdessen verwenden SOTA-Lösungen die Integration von **Jito Bundles**.[59] Jito erlaubt die atomare Bündelung von bis zu 5 Transaktionen, die direkt an die Jito-Block-Engine gesendet werden.[4] Das Bundle garantiert eine "Alles-oder-Nichts"-Ausführung: Wenn ein Arbitrage-Leg scheitert oder die vordefinierte Slippage durchbrochen wird, wird die gesamte Sequenz revertiert, was das Kapital des Traders schützt.[62] Zusätzlich kann die Direktive jitodontfront integriert werden, um Sandwich-Eingriffe zu untersagen.[62]

8.2 Dynamische Tip-Kalibrierung

Ein wiederkehrendes Muster für gescheiterte Trading-Agenten sind statische Jito-Tips (z. B. konstant 0,0001 SOL pro Bundle).[28] Bei hoher Netzwerkaktivität werden Bundles mit zu geringem Tip von den Suchern (Searchers) stummschweigend nach unten priorisiert und laufen aus ("Drop"), während konkurrierende Bots den Trade gewinnen.[28] SOTA Rust-Bots integrieren eine dynamische Tip-Kalibrierung (Transaction Cost Awareness).[28] Der Bot schätzt die Profitabilität der Transaktion in Echtzeit und bietet dem Validator dynamisch zwischen 50 % und

60 % des erwarteten Gewinns als Gebühr an (inklusive eines Floors und Ceilings), um die Inklusion in den aktuellen Block wirtschaftlich zu garantieren.[4]

9. Zusammenfassung und strategische Empfehlungen

Die Evaluierung der offiziellen Dokumentationen von Chainstack und Triton One sowie tiefgreifender Code-Repositories und Berichte der quantitativen Solana-Community zeichnet ein klares Bild: Die Extraktion und Analyse von SPL-Token hat sich von einer simplen Abfragearchitektur zu einer ultra-niedriglatenten Pipeline auf Enterprise-Niveau gewandelt.[7]

Die absolute SOTA-Lösung stützt sich auf vier unzertrennliche Säulen:

1. **Infrastruktur & Ingest:** Die Verwendung dedizierter, Kolokations-gestützter Bare-Metal-Knoten mit DPDK und die Ablösung von RPC/WebSockets durch **Yellowstone gRPC (Dragon's Mouth)**. Der Zwang, **Jito ShredStream** zu nutzen, um Turbine zu umgehen und Block-Shreds in dem Moment abzugreifen, in dem sie vom Leader generiert werden, sichert den entscheidenden Latenzvorteil von durchschnittlich 32 Millisekunden.[3] Provider wie Chainstack bieten hierbei durch ihre Jito-Integration und das Flat-Fee (Unlimited Events) gRPC-Preismodell ökonomische Planungssicherheit.[19]
2. **Verarbeitung:** Der Einsatz von Allokations-freien **Rust-Pipelines** mit Lock-Free MPMC-Channels. Die Dekodierung der Base58-Nutzlasten erfolgt typsicher in Echtzeit durch das **Yellowstone Vixen** Framework, wodurch Programm-States und Token-Metadaten direkt interpretiert werden.[39]
3. **Intelligence:** Das Generieren feingranularer Features – insbesondere das Aufspüren von 1-Hop Whale-Clustern als Frühindikator.[49] Diese Metriken fließen über Message-Broker und ClickHouse-Aggregate in rekurrente neuronale Netze, bei denen das **GRU-Modell** nachweislich die präzisesten Sub-Prozent-Vorhersagen (MAPE) auf zeitkritischen Krypto-Märkten liefert.[9]
4. **Ausführung:** Die direkte, deterministische Exekution der algorithmischen Handelsentscheidungen über **Jito Bundles** mit dynamischer Tip-Kalibrierung zur Vermeidung von MEV-Attacken und zur Absicherung der Block-Inklusion.[28]

In Summe manifestiert diese Architektur nicht nur den neuesten Stand der Technik, sondern bildet die zwingende infrastrukturelle Grundbedingung, um auf dem schnellsten und kompetitivsten Kryptomarkt der Welt im Jahr 2026 profitabel algorithmische Analysen und Vorhersagen durchzuführen.

Referenzen

1. 10 Solana infrastructure providers compared: What I found after testing them in 2026 | by Daniel Yavorovych - Medium, Zugriff am April 7, 2026, https://medium.com/@yavorovych/10-solana-infrastructure-providers-compared-what-i-found-after-testing-them-in-2026-b026970c6ccd
2. Top 10 Solana Infrastructure Providers in 2026 - RPC Fast, Zugriff am April 7, 2026,

https://rpcfast.com/blog/solana-infrastructure-providers
3. Complete guide to Solana streaming and Yellowstone gRPC - Triton Blog, Zugriff am April 7, 2026, https://blog.triton.one/complete-guide-to-solana-streaming-and-yellowstone-grpc/
4. How To Build Low-Latency Infrastructure For Solana AI Agents | RPC Fast, Zugriff am April 7, 2026, https://rpcfast.com/blog/how-to-build-low-latency-infrastructure-for-solana-ai-agents
5. Solana MEV infrastructure: why the stack matters more than the strategy | by Daniel Yavorovych - Medium, Zugriff am April 7, 2026, https://medium.com/@yavorovych/solana-mev-infrastructure-why-the-stack-matters-more-than-the-strategy-7a647e248924
6. Real-time Solana data: WebSocket subscriptions vs Yellowstone gRPC Geyser - Chainstack, Zugriff am April 7, 2026, https://chainstack.com/real-time-solana-data-websocket-vs-yellowstone-grpc-geyser/
7. Stop polling Solana: The CTO's guide to data streaming and gRPC | by Daniel Yavorovych | Mar, 2026 | Medium, Zugriff am April 7, 2026, https://medium.com/@yavorovych/stop-polling-solana-the-ctos-guide-to-data-streaming-and-grpc-021ba2c6a747
8. Low-Latency Solana Playbook for HFT | RPCFast, Zugriff am April 7, 2026, https://docs.rpcfast.com/solana-dedicated-nodes/low-latency-solana-playbook-for-hft
9. Development of a cryptocurrency price prediction model: leveraging GRU and LSTM for Bitcoin, Litecoin and Ethereum - PMC, Zugriff am April 7, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC11935774/
10. Jito ShredStream integrations hints for Solana HFT teams | by Daniel Yavorovych - Medium, Zugriff am April 7, 2026, https://yavorovych.medium.com/jito-shredstream-integrations-hints-for-solana-hft-teams-cb6adf852a85
11. Solana Geyser Plugins: Powering High-Speed Data Streaming Guide | by Yong kang Chia, Zugriff am April 7, 2026, https://extremelysunnyyk.medium.com/solana-geyser-plugins-powering-high-speed-data-streaming-guide-9ae2328b8454
12. Monitor Solana Programs with Yellowstone Geyser gRPC (TypeScript) | Quicknode Guides, Zugriff am April 7, 2026, https://www.quicknode.com/guides/solana-development/tooling/geyser/yellowstone
13. Solana Geyser Plugins: Streaming Data at the Speed of Light - Helius, Zugriff am April 7, 2026, https://www.helius.dev/blog/solana-geyser-plugins-streaming-data-at-the-speed-of-light
14. Solana MEV infrastructure: why the stack matters more than the strategy, Zugriff am April 7, 2026,

https://yavorovych.medium.com/solana-mev-infrastructure-why-the-stack-matters-more-than-the-strategy-7a647e248924
15. chainstacklabs/grpc-geyser-tutorial: This repository provides Python examples that show how to get live updates from the Solana blockchain using the Yellowstone gRPC plugin - GitHub, Zugriff am April 7, 2026, https://github.com/chainstacklabs/grpc-geyser-tutorial
16. Triton's Dragon's Mouth Yellowstone gRPC service for high-performance Solana streaming - GitHub, Zugriff am April 7, 2026, https://github.com/rpcpool/yellowstone-grpc
17. Yellowstone gRPC Upgrade: Faster Solana Data & Expanded Monitoring with Velocity Tier, Zugriff am April 7, 2026, https://blog.quicknode.com/even-faster-solana-data-with-yellowstone-grpc/
18. Solana Yellowstone gRPC: Real-Time Data Streaming - Helius Docs, Zugriff am April 7, 2026, https://www.helius.dev/docs/grpc
19. Yellowstone gRPC Geyser plugin - Chainstack Docs, Zugriff am April 7, 2026, https://docs.chainstack.com/docs/yellowstone-grpc-geyser-plugin
20. Solana Pump AMM Data Streaming: gRPC Guide - Helius Docs, Zugriff am April 7, 2026, https://www.helius.dev/docs/grpc/stream-pump-amm-data
21. How to use the Solana Geyser plugin to stream data with Yellowstone gRPC - Chainstack, Zugriff am April 7, 2026, https://chainstack.com/how-to-use-the-solana-geyser-plugin-to-stream-data-with-yellowstone-grpc/
22. Crude example of a gRPC/WebSocket-based Solana token price monitor. - GitHub Gist, Zugriff am April 7, 2026, https://gist.github.com/rubpy/6b474669a4ddb82277edaaf3f1748b59
23. Winning the Millisecond Game: Shreds, LaserStream, and the Edge of Solana - Helius, Zugriff am April 7, 2026, https://www.helius.dev/blog/solana-shreds
24. How to improve Solana RPC latency with ShredStream | Chainstack Blog, Zugriff am April 7, 2026, https://chainstack.com/how-to-improve-solana-rpc-latency-with-shredstream/
25. Low-latency Solana playbook for HFT traders - RPC Fast, Zugriff am April 7, 2026, https://rpcfast.com/blog/low-latency-solana-playbook-hft-traders
26. Shred vs GRPC Performance Benchmark - GitHub, Zugriff am April 7, 2026, https://github.com/fairrustana/shredstream-or-grpc
27. What's the best way to build a low-latency dedicated gRPC node - Solana Stack Exchange, Zugriff am April 7, 2026, https://solana.stackexchange.com/questions/23944/whats-the-best-way-to-build-a-low-latency-dedicated-grpc-node
28. What a Solana MEV bot looks like when the infrastructure is wrong - Troop Messenger, Zugriff am April 7, 2026, https://www.troopmessenger.com/blogs/solana-mev-bot-wrong-infrastructure
29. Understanding Solana Data Streams and Protocols (Shreds, gRPC, WS, UDP) | by kishi.sol, Zugriff am April 7, 2026, https://medium.com/@KishiTheMechanic/understanding-solana-data-streams-and-protocols-shreds-grpc-ws-udp-da2f35a7dab4

30. Solana RPC Nodes—The Complete Guide, Zugriff am April 7, 2026, https://rpcfast.com/blog/solana-rpc-node-full-guide
31. Optimizing Solana RPC Performance with Hardware Tuning | by Alex Carter - Medium, Zugriff am April 7, 2026, https://medium.com/@alexcarter98zr/optimizing-solana-rpc-performance-with-hardware-tuning-1a1c0cb55fc4
32. Linux Kernel vs DPDK: HTTP Performance Showdown | talawah.io, Zugriff am April 7, 2026, https://talawah.io/blog/linux-kernel-vs-dpdk-http-performance-showdown/
33. Kernel bypass - The Cloudflare Blog, Zugriff am April 7, 2026, https://blog.cloudflare.com/kernel-bypass/
34. Best Solana RPC providers for fast and reliable production in 2026 - Chainstack, Zugriff am April 7, 2026, https://chainstack.com/best-solana-rpc-providers-in-2026/
35. Helius RPC provider overview (2026) | Chainstack Blog, Zugriff am April 7, 2026, https://chainstack.com/helius-rpc-provider-a-practical-overview/
36. Best Base RPC providers for onchain applications in 2026 - Chainstack, Zugriff am April 7, 2026, https://chainstack.com/base-rpc-providers-2026/
37. Solana: Listening to programs using Geyser and Yellowstone gRPC in Node.js - Chainstack, Zugriff am April 7, 2026, https://docs.chainstack.com/docs/solana-listening-to-programs-using-geyser-and-yellowstone-grpc-node-js
38. Solana gRPC Best Practices - Shyft Docs, Zugriff am April 7, 2026, https://docs.shyft.to/solana-yellowstone-grpc/solana-grpc-best-practices
39. solana yellowstone geyser grpc with javascript, Zugriff am April 7, 2026, https://solana.stackexchange.com/questions/22909/solana-yellowstone-geyser-grpc-with-javascript
40. gRPC streaming is the most difficult thing I have ever tried in crypto : r/solana - Reddit, Zugriff am April 7, 2026, https://www.reddit.com/r/solana/comments/1jspqw8/grpc_streaming_is_the_most_difficult_thing_i_have/
41. Rust High Frequency Trading - Design Decisions : r/rust - Reddit, Zugriff am April 7, 2026, https://www.reddit.com/r/rust/comments/1ij7ldn/rust_high_frequency_trading_design_decisions/
42. Engineering Solana Trading Bots: 2026 Infrastructure Guide - Dysnix, Zugriff am April 7, 2026, https://dysnix.com/blog/solana-trading-bot-guide
43. How to Build a Lock-Free Data Structure in Rust - OneUptime, Zugriff am April 7, 2026, https://oneuptime.com/blog/post/2026-01-30-how-to-build-a-lock-free-data-structure-in-rust/view
44. sof - crates.io: Rust Package Registry, Zugriff am April 7, 2026, https://crates.io/crates/sof/range/%5E0.18.1
45. Indexing - Solana, Zugriff am April 7, 2026, https://solana.com/docs/payments/accept-payments/indexing

46. rpcpool/yellowstone-vixen: Solana program parsing toolkit - GitHub, Zugriff am April 7, 2026, https://github.com/rpcpool/yellowstone-vixen
47. Solana Token-2022 metadata: from conventions to state - Chainstack, Zugriff am April 7, 2026, https://chainstack.com/solana-token-2022-metadata-from-conventions-to-explicit-state/
48. Streaming accurate mint price based on transaction data with yellowstone gRPC, Zugriff am April 7, 2026, https://solana.stackexchange.com/questions/22489/streaming-accurate-mint-price-based-on-transaction-data-with-yellowstone-grpc
49. Whale positioning data shows SOL is the only major token being accumulated right now, Zugriff am April 7, 2026, https://www.reddit.com/r/CryptoMarkets/comments/1rvklf3/whale_positioning_data_shows_sol_is_the_only/
50. Next-Gen Whale Monitoring Feed: Exposing the Hidden Moves of Solana Memecoin Whales | by Nitin Jakhar | Medium, Zugriff am April 7, 2026, https://medium.com/@nitin132/next-gen-whale-monitoring-feed-exposing-the-hidden-moves-of-solana-memecoin-whales-c52d8dc322e2
51. Getting new Solana tokens in real time?, Zugriff am April 7, 2026, https://solana.stackexchange.com/questions/20626/getting-new-solana-tokens-in-real-time
52. Causal Feature Engineering of Price Directions of Cryptocurrencies using Dynamic Bayesian Networks - arXiv, Zugriff am April 7, 2026, https://arxiv.org/html/2306.08157v2
53. High-Frequency Cryptocurrency Price Forecasting Using Machine Learning Models: A Comparative Study - MDPI, Zugriff am April 7, 2026, https://www.mdpi.com/2078-2489/16/4/300
54. An Integrated Framework for Cryptocurrency Price Forecasting and Anomaly Detection Using Machine Learning - MDPI, Zugriff am April 7, 2026, https://www.mdpi.com/2076-3417/15/4/1864
55. Machine Learning-Based Approach for Predicting the Altcoins Price Direction Change from a High-Frequency Data of Seven Years Based on Socio-Economic Factors, Bitcoin Prices, Twitter and News Sentiments - IDEAS/RePEc, Zugriff am April 7, 2026, https://ideas.repec.org/a/kap/compec/v64y2024i5d10.1007_s10614-023-10538-5.html
56. 7 Solana Indexing Stacks That Stay Real-Time | by Nexumo | Medium, Zugriff am April 7, 2026, https://medium.com/@Nexumo_/7-solana-indexing-stacks-that-stay-real-time-399b1f8c89db
57. How to Build a Real-Time Analytics Dashboard with ClickHouse - OneUptime, Zugriff am April 7, 2026, https://oneuptime.com/blog/post/2026-01-21-clickhouse-realtime-analytics-dashboard/view
58. hodlwarden/solana-arbitrage-bot - GitHub, Zugriff am April 7, 2026,

https://github.com/hodlwarden/solana-arbitrage-bot
59. cutupdev/Solana-Pumpfun-Sniper-Bot - GitHub, Zugriff am April 7, 2026, https://github.com/cutupdev/Solana-Pumpfun-Sniper-Bot
60. Building Production-Grade Solana Sniper Bots: A 2026 Technical Blueprint | Dysnix, Zugriff am April 7, 2026, https://dysnix.com/blog/complete-stack-competitive-solana-sniper-bots
61. MEV Protection on Solana in 2026 — Jito Bundles, Astralane, and What Actually Works, Zugriff am April 7, 2026, https://dev.to/gerus_team/mev-protection-on-solana-in-2026-jito-bundles-astralane-and-what-actually-works-3gbc
62. Solana Trading Infrastructure - Full Guide in 2026 - RPC Fast, Zugriff am April 7, 2026, https://rpcfast.com/blog/solana-trading-infrastructure
63. How to get a Solana RPC endpoint in 2026 | Chainstack Blog, Zugriff am April 7, 2026, https://chainstack.com/how-to-get-solana-rpc-endpoint-in-2026/